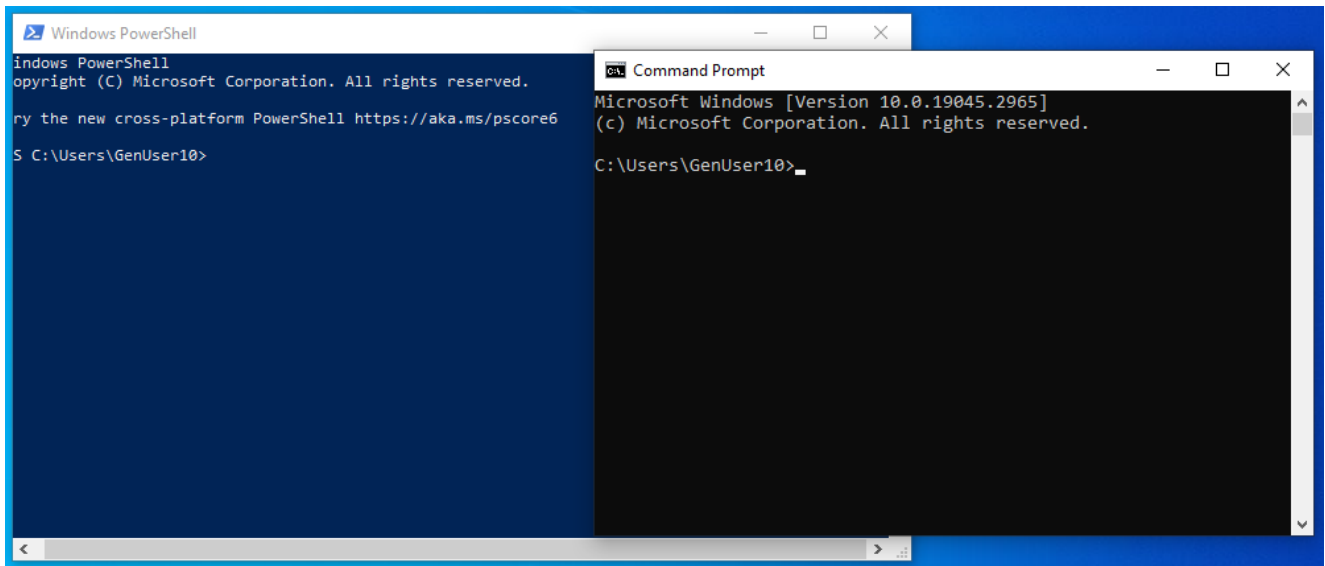
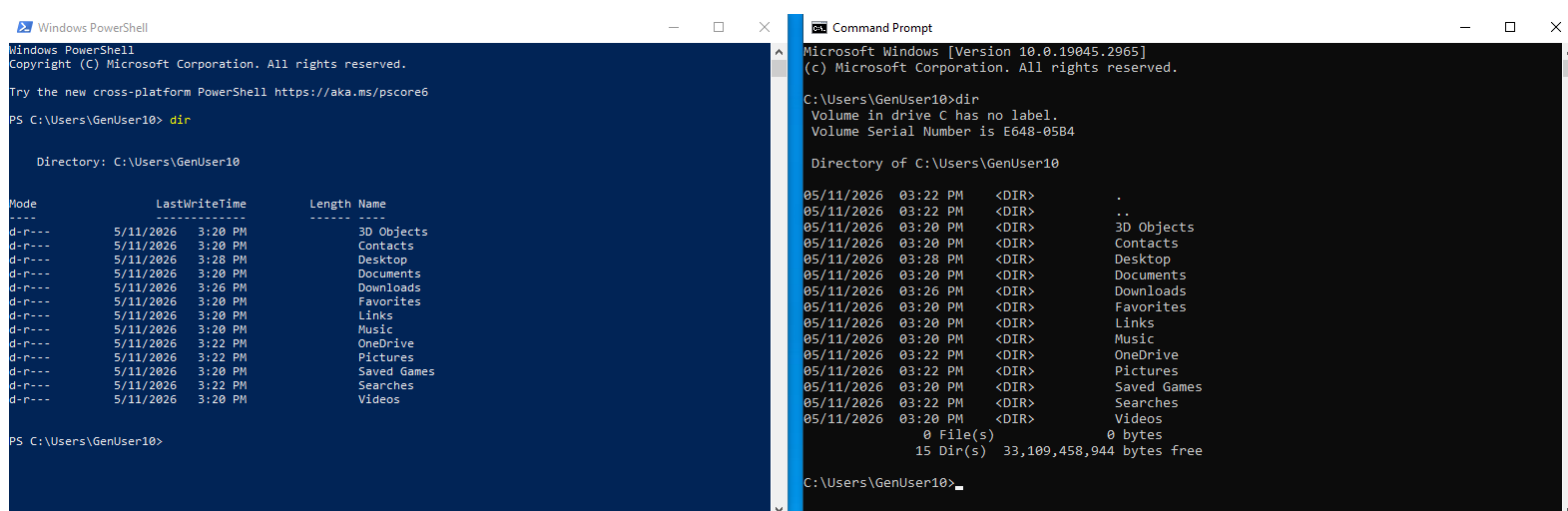


POWERSHELL:

Breakdown del funzionamento di powershell, comandi, comandlets e funzionalità di base come scripting.



A differenza del Command Prompt (cmd.exe) powershell non è un semplice terminale/interprete di comandi CLI bensì è un framework scripting e di automazione, object-oriented e basato su .NET di Microsoft. Proviamo il Comando “dir”



Vediamo una enumerazione delle directories sul computer host in entrambi i casi ma l'output è leggermente diverso, powershell ci dà una lista di perms mentre cmd ci restituisce solo il nome delle directories, ma in più ci informa dei byte di spazio libero e del numero directories, questo perché in sé non sono gli stessi comandi, dir è un alias di un comandlet di formato “Verb-Object” powershell proprio. Andiamo a vedere.

dir quindi è alias del
comandlet “Get-Alias” powershell
con una funzionalità simile.
Ma cos'è un comandlet?
Come riportato da documentazione
powershell di Microsoft stessa, i
comandlets sono comandi leggeri
di ambiente powershell che svolgono
un'azione che restituisce oggetto
di Microsoft .NET (OOP) e che hanno

```
PS C:\Users\GenUser10> Get-Alias dir

CommandType      Name
-----
Alias             dir -> Get-ChildItem

PS C:\Users\GenUser10> Get-ChildItem

Directory: C:\Users\GenUser10

Mode                LastWriteTime         Length Name
----                -
d-r---          5/11/2026   3:20 PM             3D Objects
d-r---          5/11/2026   3:20 PM             Contacts
d-r---          5/11/2026   3:28 PM             Desktop
d-r---          5/11/2026   3:20 PM             Documents
d-r---          5/11/2026   3:26 PM             Downloads
d-r---          5/11/2026   3:20 PM             Favorites
d-r---          5/11/2026   3:20 PM             Links
d-r---          5/11/2026   3:20 PM             Music
d-r---          5/11/2026   3:22 PM             OneDrive
d-r---          5/11/2026   3:22 PM             Pictures
d-r---          5/11/2026   3:20 PM             Saved Games
d-r---          5/11/2026   3:22 PM             Searches
d-r---          5/11/2026   3:20 PM             Videos
```

formato Verbo-Object che li definiscono e rendono il codice leggibile, accessibile, intuitivo e ottimo per automazione di task su enviroment Windows.

Informazioni generali sui cmdlet

Un cmdlet è un comando leggero usato nell'ambiente PowerShell. Il runtime di PowerShell richiama questi cmdlet all'interno del contesto degli script di automazione forniti nella riga di comando. Il runtime di PowerShell li richiama anche a livello di codice tramite le API di PowerShell.

Cmdlets

I cmdlet eseguono un'azione e in genere restituiscono un oggetto Microsoft .NET al comando successivo nella pipeline. Un cmdlet è un singolo comando che partecipa alla semantica della pipeline di PowerShell. Sono inclusi cmdlet binari (C#), funzioni di script avanzate, CDXML e flussi di lavoro.

Nomi dei cmdlet

PowerShell usa una coppia di nomi verbi e sostantivi per denominare i cmdlet.

Comando netstat: è il tool built-in Windows per mostrare connessione TCP/UDP attive, port con servizi in ascolto e statistiche di protocollo o routing table. Come spiegato anche dal comando netstat -h (help). Vediamo le flags di interesse.

```
PS C:\Users\GenUser10> netstat -h

Displays protocol statistics and current TCP/IP network connections.

NETSTAT [-a] [-b] [-e] [-f] [-n] [-o] [-p proto] [-r] [-s] [-t] [-x] [-y] [interval]

-a          Displays all connections and listening ports.
-b          Displays the executable involved in creating each connection or
           listening port. In some cases well-known executables host
           multiple independent components, and in these cases the
           sequence of components involved in creating the connection
           or listening port is displayed. In this case the executable
           name is in [] at the bottom, on top is the component it called,
           and so forth until TCP/IP was reached. Note that this option
           can be time-consuming and will fail unless you have sufficient
           permissions.
-e          Displays Ethernet statistics. This may be combined with the -s
           option.
-f          Displays Fully Qualified Domain Names (FQDN) for foreign
           addresses.
-n          Displays addresses and port numbers in numerical form.
-o          Displays the owning process ID associated with each connection.
-p proto    Shows connections for the protocol specified by proto; proto
           may be any of: TCP, UDP, TCPv6, or UDPv6. If used with the -s
           option to display per-protocol statistics, proto may be any of:
           IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP, or UDPv6.
-q          Displays all connections, listening ports, and bound
           nonlistening TCP ports. Bound nonlistening ports may or may not
           be associated with an active connection.
-r          Displays the routing table.
-s          Displays per-protocol statistics. By default, statistics are
           shown for IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP, and UDPv6;
           the -p option may be used to specify a subset of the default.
-t          Displays the current connection offload state.
-x          Displays NetworkDirect connections, listeners, and shared
           endpoints.
-y          Displays the TCP connection template for all connections.
           Cannot be combined with the other options.
interval    Redisplays selected statistics, pausing interval seconds
           between each display. Press CTRL+C to stop redisplaying
           statistics. If omitted, netstat will print the current
           configuration information once.
```

Per questo esercizio vedremo la flag -r per ottenere il routing table della nostra VM, e infine -abno (tutte le connessioni, tutti i binaries correlati alla connessione, indirizzo ip e port number, PID associato)

IPv4 Route Table				
=====				
Active Routes:				
Network	Destination	Netmask	Gateway	Interface
	0.0.0.0	0.0.0.0	10.0.2.2	10.0.2.15
	10.0.2.0	255.255.255.0	On-link	10.0.2.15
	10.0.2.15	255.255.255.255	On-link	10.0.2.15
	10.0.2.255	255.255.255.255	On-link	10.0.2.15
	127.0.0.0	255.0.0.0	On-link	127.0.0.1
	127.0.0.1	255.255.255.255	On-link	127.0.0.1
	127.255.255.255	255.255.255.255	On-link	127.0.0.1
	224.0.0.0	240.0.0.0	On-link	127.0.0.1
	224.0.0.0	240.0.0.0	On-link	10.0.2.15
	255.255.255.255	255.255.255.255	On-link	127.0.0.1
	255.255.255.255	255.255.255.255	On-link	10.0.2.15
=====				

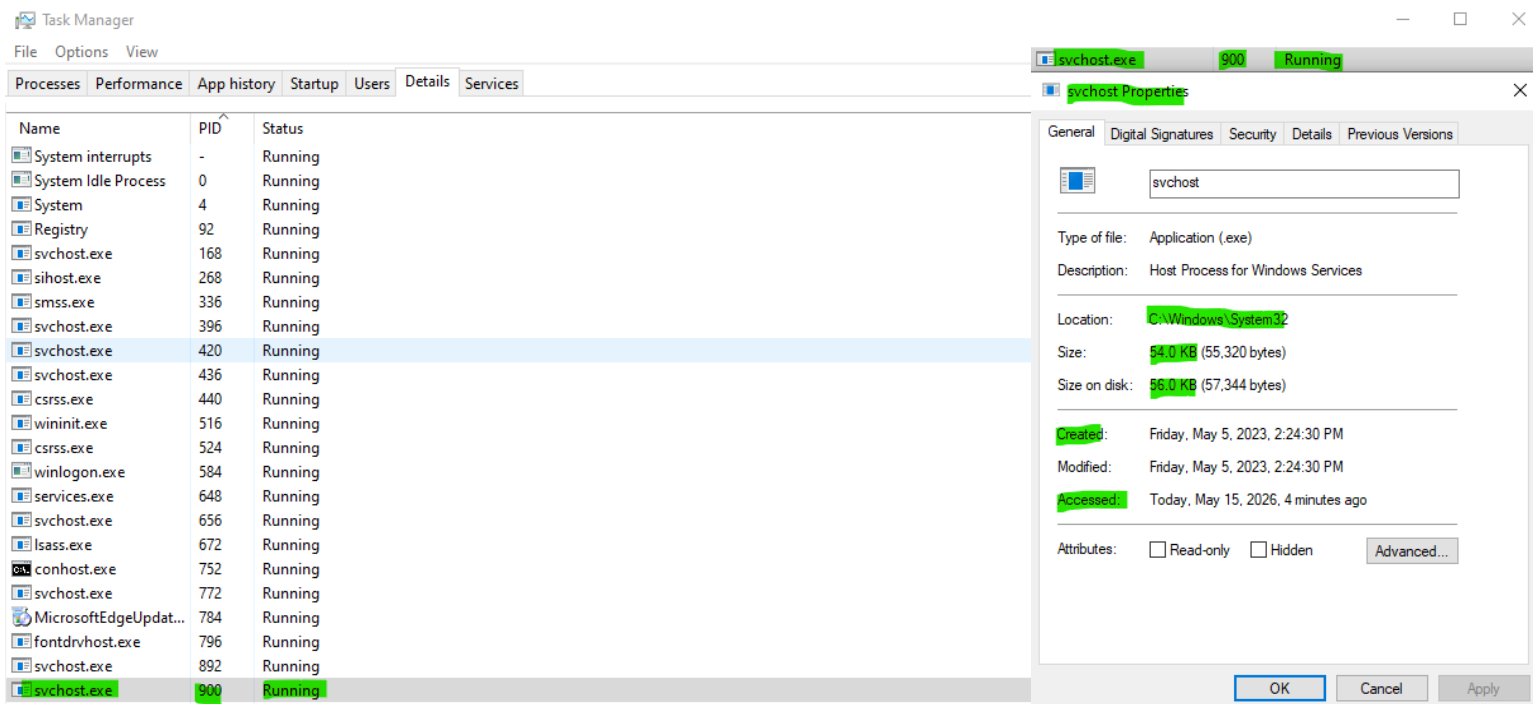
0.0.0.0 è l'equivalente IPv4 di "outbound" ovvero al di fuori della rete locale, da questo possiamo quindi estrapolare il Gateway necessario per le comunicazioni tra LAN interna e WAN esterna (o nel caso di una VM della mia LAN di casa personale e successivamente del WAN internet/cloud/ISP), in questo caso vediamo come il Gateway di default sia 10.0.2.2 nella stessa interfaccia di rete NAT configurata da Virtualbox per 10.0.2.15 (Ipv4 Host). Infine 10.0.2.255 è l'ip di broadcast del subnet nel range IPv4 privato dell'host. 127.0.0.0 è IPv4 di loop locale, 127.0.0.1 è l'ipv4 di localhost stesso della macchina, infine 127.255.255.255 è per il broadcast locale. 224.0.0.0 su interfacce di localhost e IP privato sono per il Multicast tra le reti della macchina. Infine 255.255.255.255 è il broadcast generale

Con netstat -abno ottengo una list di connessioni attiva con protocollo usato, indirizzo source/destination, stato della porta e PID (process ID). Prendiamo per esempio il PID 900, connessione TCP RpcSs avviata dal binario/executable svchost.exe

Possiamo vedere su task manager tutte le informazioni principali riguardanti questo servizio attivo.

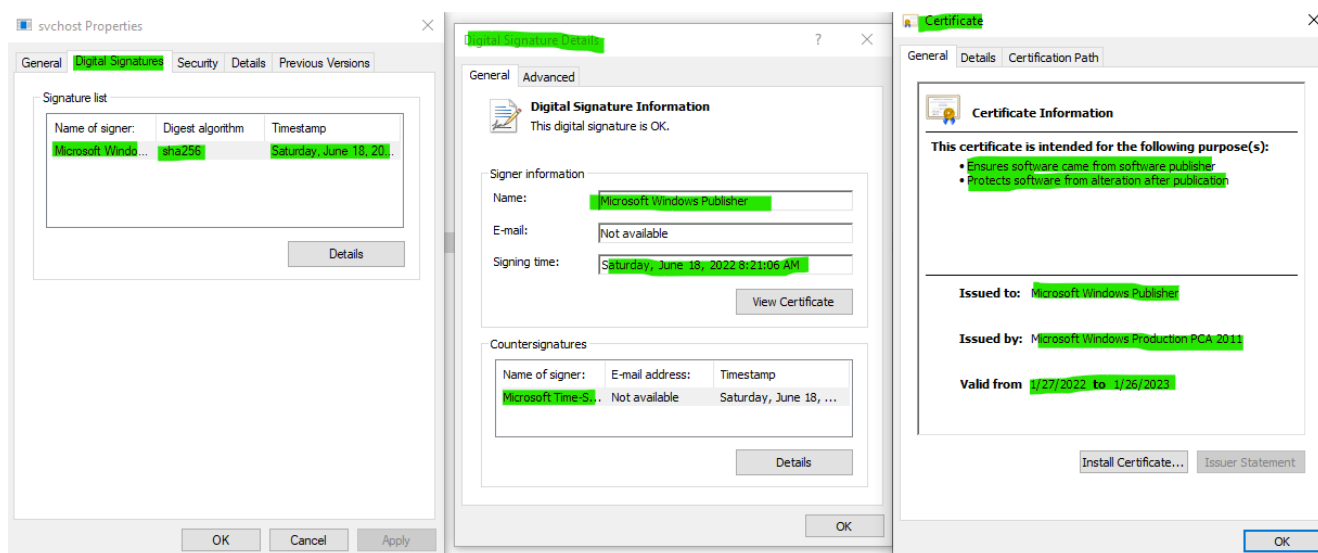
```
PS C:\Windows\system32> netstat -abno
```

Active Connections				
Proto	Local Address	Foreign Address	State	PID
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING	900
RpcSs				
[svchost.exe]				
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING	4
Can not obtain ownership information				
TCP	0.0.0.0:5040	0.0.0.0:0	LISTENING	396
CDPSvc				
[svchost.exe]				
TCP	0.0.0.0:7680	0.0.0.0:0	LISTENING	6092
Can not obtain ownership information				
TCP	0.0.0.0:49664	0.0.0.0:0	LISTENING	672
[lsass.exe]				
TCP	0.0.0.0:49665	0.0.0.0:0	LISTENING	516
Can not obtain ownership information				
TCP	0.0.0.0:49666	0.0.0.0:0	LISTENING	1160
EventLog				
[svchost.exe]				
TCP	0.0.0.0:49667	0.0.0.0:0	LISTENING	1112
Schedule				
[svchost.exe]				
TCP	0.0.0.0:49668	0.0.0.0:0	LISTENING	2308
[spoolsv.exe]				
TCP	0.0.0.0:49669	0.0.0.0:0	LISTENING	648
Can not obtain ownership information				
TCP	10.0.2.15:139	0.0.0.0:0	LISTENING	4
Can not obtain ownership information				
TCP	10.0.2.15:49675	20.44.10.122:443	TIME_WAIT	0
TCP	10.0.2.15:49676	23.206.246.162:80	ESTABLISHED	168
WpnUserService_3d545				
[svchost.exe]				
TCP	10.0.2.15:49677	184.25.52.64:443	ESTABLISHED	168
WpnUserService_3d545				

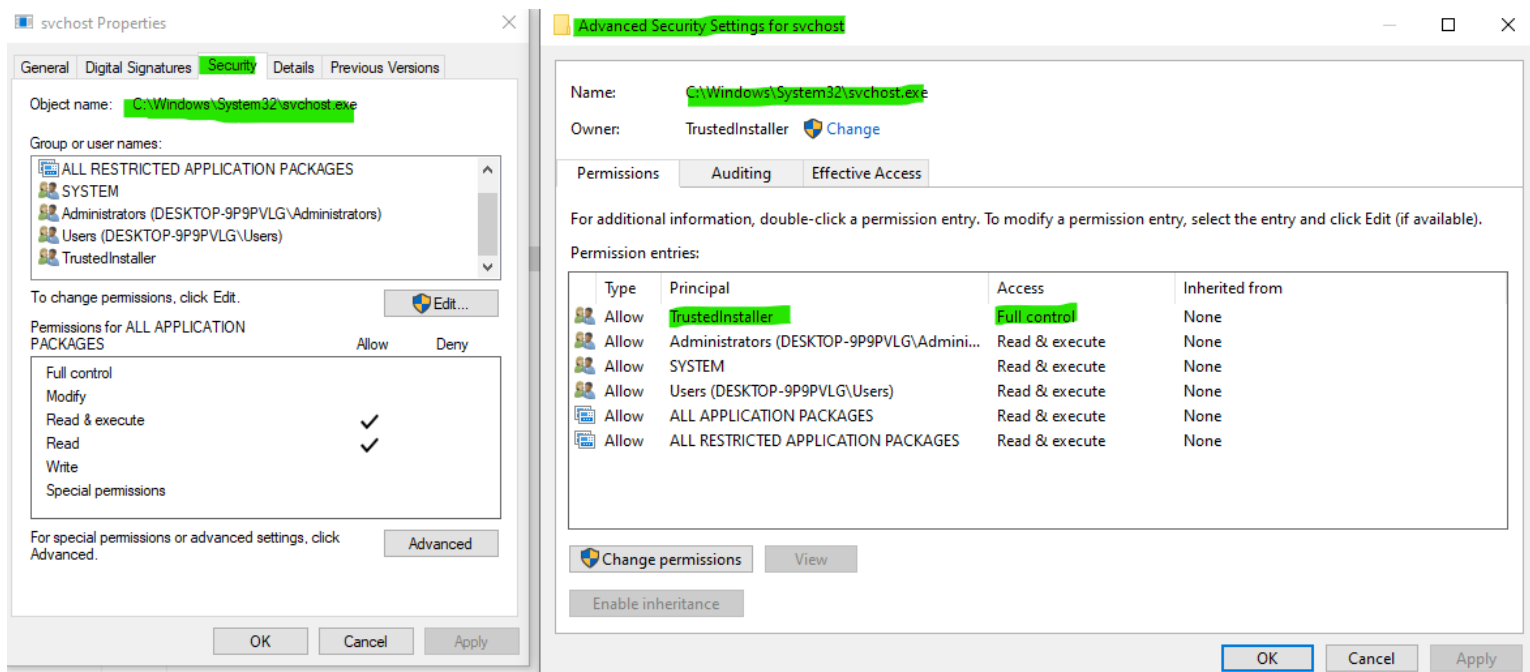


Click con tasto destro mouse, vediamo le proprietà. Sulla landing page generale di overview delle proprietà del servizio possiamo intanto vedere come questo si trovi su filepath C:\Windows\System32; la directory corretta per un servizio ufficiale di Windows. Size e Size su disco sembrano essere abbastanza simili, non ci sono incongruenze, secondo indicatore di essere un processo legittimo, infine vediamo una data di creazione del 2023 (data dell'ISO del creation tool, in questo caso non è sospetto) e ultimo accesso 4 minuti fa (dopo startup). Tutto quadra. Ma servono ulteriori controlli.

Vediamo le Digital Signatures: Signature di Microsoft Windows, con algoritmo SHA256, del 2022 (si tratta di un immagine ISO vecchiotta), il certificato è però scaduto.

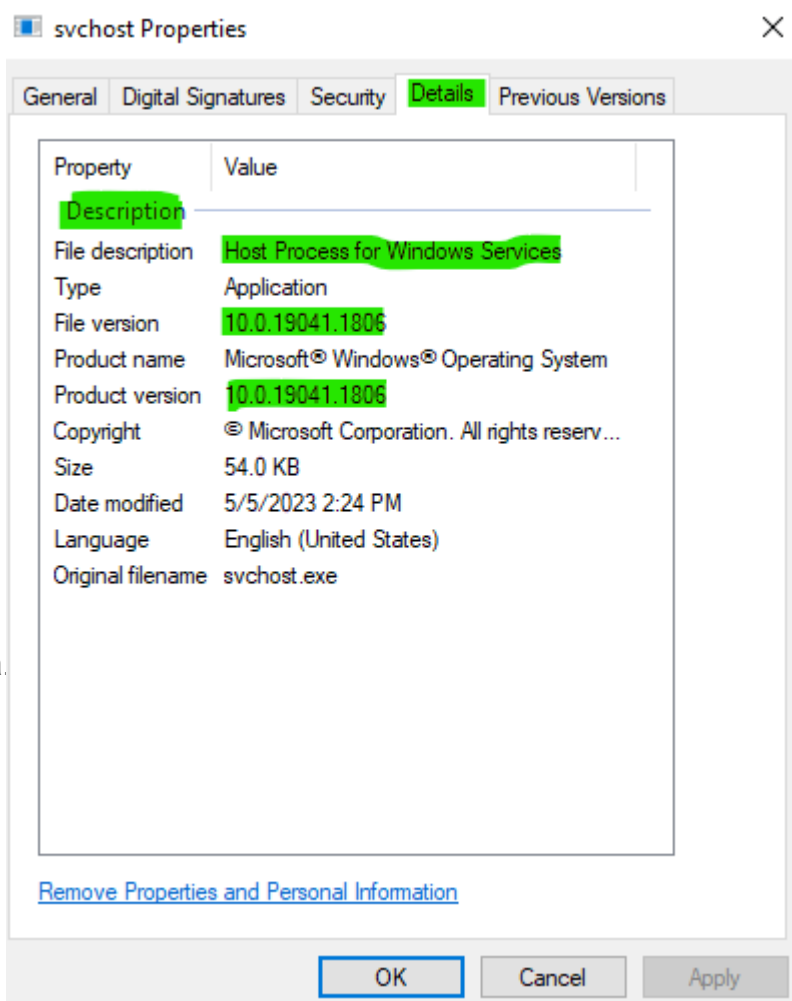


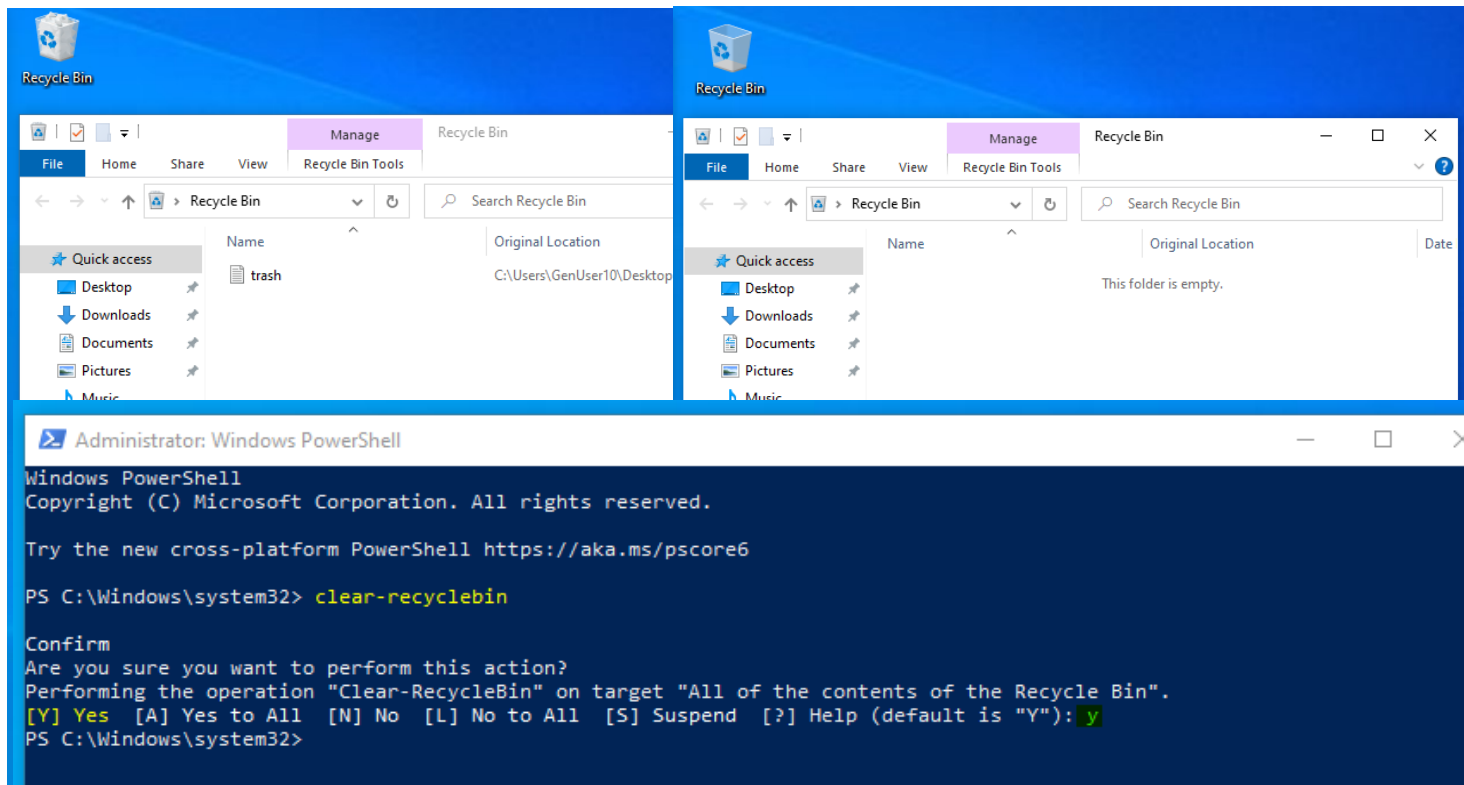
Poiché la VM ISO image è abbastanza datata per ora è tutto nella norma, si tratta di una clean installation di Win10 Pro è normale che i certificati siano scaduti.



Nelle advanced security settings vedo come l'unico utente a poter agire su questo servizio/file sia un utente di sistema TrustedInstaller, SYSTEM potrebbe prenderne ownership ma in questo caso non ne ha bisogno. Assenza di incongruenze o anomalie mi dà ulteriore sicurezza che si tratti allora di un processo legittimo.

Da details vedo descrizione, è un Host process di Windows, ci informa della versione del file e del prodotto, ultima modifica applicata, linguaggio, nome file originale, copyright, dimensioni. Tutto combacia, è quasi sicuramente un processo legittimo (in casi di infiltrazione o di exploit servizi legittimi come questo possono comunque svolgere azioni malevoli, bisogna in quel caso avere un'immagine maggiore delle operazioni svolte sull'host tramite un'analisi dinamica).





Cancellare il Recycle Bin con powershell

AUTOMAZIONE CON CMDLET E SCRIPTS DI POWERSHELL IN CYBERSECURITY:

Mostrerò ora alcuni comandlets di base che un analista di cybersecurity potrebbe utilizzare per svolgere controlli su connessioni, servizi, applicazioni startup, scheduled tasks e signature Hash di un file.

```
PS C:\Windows\system32> Get-Process | Select-Object Id, Name, Path | Sort-Object Id
```

Id	Name	Path
0	Idle	
4	System	
92	Registry	
168	svchost	C:\Windows\system32\svchost.exe
268	sihost	C:\Windows\system32\sihost.exe
336	smss	
396	svchost	C:\Windows\system32\svchost.exe
440	csrss	

Prendi object "process" e display su schermo | pipe risultato e seleziona di process gli oggetti ID, NOME e PATH FILE da mostrare su schermo | per ordinare ID in maniera crescente fail il sort di quest'ultimo oggetto

```
900 svchost C:\Windows\system32\svchost.exe
956 svchost C:\Windows\system32\svchost.exe
```

Simile a ciò che abbiamo visto su task manager nella tab Details

La “lista” servizi windows contiene anche i servizi interrotti o scheduled o disattivi quindi noi istruiamo a powershell di:

```
PS C:\Windows\system32> Get-Service | Where-Object { $_.Status -eq "Running"}
```

Status	Name	DisplayName
Running	Appinfo	Application Information
Running	AudioEndpointBu...	Windows Audio Endpoint Builder
Running	Audiosrv	Windows Audio

Prendi i Servizi | Pipe del risultato e prendi solo “Dove l’oggetto” { servizio trovato ha status “running”}

Specifichiamo invece solo i programmi di avvio durante startup.

```
PS C:\Windows\system32> Get-CimInstance Win32_StartupCommand | Select-Object Name, Command, Location
```

Name	Command	Location
OneDriveSetup	C:\Windows\SysWOW64\OneDriveSetup.exe /thfirstsetup	HKU\S-1-5-19\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
OneDriveSetup	C:\Windows\SysWOW64\OneDriveSetup.exe /thfirstsetup	HKU\S-1-5-20\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
OneDrive	"C:\Users\GenUser10\AppData\Local\Microsoft\OneDrive\OneDrive.exe" /background	HKU\S-1-5-21-1346727324-3185195036-3653506967-1001\SOFTWARE\Microso...
SecurityHealth	%windir%\system32\SecurityHealthSystray.exe	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
VBoxTray	%SystemRoot%\system32\VBoxTray.exe	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run

Prendi tutti gli oggetti (istanze ottenute con CIM) della classe Win32_StartupCommand (app di avvio) | pipe del risultato e per ognuna selezioni gli oggetti “Nome, Comando (come li filepath in questo caso, il comando che effettivamente avvia l’applicazione) e la location della chiave nel Registry di windows

Vediamo ora le scheduled tasks:

```
PS C:\Windows\system32> Get-ScheduledTask | Where-Object { $_.State -eq "Ready" } | Select-Object TaskName, TaskPath
```

TaskName	TaskPath
MicrosoftEdgeUpdateTaskMachineUA	\
OneDrive Reporting Task-S-1-5-21-1346727324-3185195036-3653506967-1001	\
OneDrive Standalone Update Task-S-1-5-21-1346727324-3185195036-3653506967-1001	\
OneDrive Startup Task-S-1-5-21-1346727324-3185195036-3653506967-1001	\
.NET Framework NGEN v4.0.30319	\Microsoft\Windows\.NET Framework\
.NET Framework NGEN v4.0.30319 64	\Microsoft\Windows\.NET Framework\
AD RMS Rights Policy Template Management (Manual)	\Microsoft\Windows\Active Directory Rights Management Services Client\
EDP Policy Manager	\Microsoft\Windows\AppID\
Microsoft Compatibility Appraiser	\Microsoft\Windows\Application Experience\
PcaPatchDbTask	\Microsoft\Windows\Application Experience\
ProgramDataUpdater	\Microsoft\Windows\Application Experience\
StartupAppTask	\Microsoft\Windows\Application Experience\
appuriverifierdaily	\Microsoft\Windows\ApplicationData\

#Prendi oggetto “ScheduledTask” | pipe risultato e display solo dove l’oggetto scheduled task ha lo state “Ready” (pronto) | pipe ulteriore dove selezioni per tale task l’oggetto “TaskName” ovvero il nome e “TaskPath” ovvero la filepath del binario

Proviamo a prendere l'hash di un file .txt

```
PS C:\Windows\system32> Get-FileHash C:\Users\GenUser10\Desktop\test_me.txt -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	E3B0C44298FC1C149AFBF4C8996FB92427AE41E4649B934CA495991B7852B855	C:\Users\GenUser10\Desktop\test_me.txt

Prendi oggetto FileHash su path file con algoritmo SHA256

```
PS C:\Windows\system32> Get-FileHash C:\Users\GenUser10\Desktop\test_me.txt -Algorithm SHA256,SHA1,MD5
Get-FileHash : Cannot process argument transformation on parameter 'Algorithm'. Cannot convert value to type System.String.
At line:1 char:64
+ ... ash C:\Users\GenUser10\Desktop\test_me.txt -Algorithm SHA256,SHA1,MD5
+ ~~~~~
+ CategoryInfo          : InvalidData: (:) [Get-FileHash], ParameterBindingArgumentTransformationException
+ FullyQualifiedErrorId : ParameterArgumentTransformationError,Get-FileHash
```

GetFileHash però non accetta più argomenti per l'algoritmo, solo uno alla volta, posso però ora usare le capacità di scripting di powershell!

```
PS C:\Windows\system32> "SHA256","SHA1","MD5" | ForEach-Object {Get-FileHash C:\Users\GenUser10\Desktop\test_me.txt -Algorithm $_}
```

Algorithm	Hash	Path
SHA256	E3B0C44298FC1C149AFBF4C8996FB92427AE41E4649B934CA495991B7852B855	C:\Users\GenUser10\Desktop\test_me.txt
SHA1	DA39A3EE5E6B4B0D3255BFEF95601890AFD80709	C:\Users\GenUser10\Desktop\test_me.txt
MD5	D41D8CD98F00B204E9800998ECF8427E	C:\Users\GenUser10\Desktop\test_me.txt

#Dichiariamo gli oggetti SHA256, SHA1, MD5 | con ciclo For gestito da comandlet ForEach-Object {Prendi FileHash su path file(per ognuno dei 3 algoritmi)}

Molto comodo per creare IoC ad esempio!